

SLOTIFY: A SMART APPOINTMENT BOOKING SYSTEM FOR SERVICE-BASED BUSINESSES

A Project Report

Submitted By

Aksh Thakkar - 2403031057002

Om Maurya - 2403031057048

Daksh Patel - 2403031057065

Jainil Patel - 2403031057071

in Partial Fulfilment For the Award of

the Degree of

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE AND ENGINEERING

Under the Guidance of

Prof. Pratik Gite

Your Guide Designation



VADODARA

November - 2026



PARUL UNIVERSITY

CERTIFICATE

This is to Certify that Project - II (303105423) of 7th Semester entitled “Slotify: A Smart Appointment Booking System for Service-Based Businesses” of Group No. CSE_182 has been successfully completed by

- Aksh Thakkar - 2403031057002
- Om Maurya - 2403031057048
- Daksh Patel - 2403031057065
- Jainil Patel - 2403031057071

under my guidance in partial fulfillment of the Bachelor of Technology (B.Tech) in Computer Science and Engineering of Parul University in Academic Year 2026- 2027.

Date of Submission :_____

Prof. Pratik Gite

Prof. Namrata Patel

Dr. Shailendra K Mishra

Project Guide

Project Coordinator

Head of Department

CSE, PIET

CSE, PIET

CSE, PIET

Parul University

Parul University

Parul University

Acknowledgements

“The single greatest cause of happiness is gratitude.”

-Auliq-Ice

Acknowledgements

We express our sincere gratitude to all those who supported and guided us throughout the successful completion of this project. First and foremost, we are thankful to the Almighty for granting us the strength, wisdom, and determination to carry out this work.

We extend our heartfelt thanks to our project guide, **Prof. Pratik Gite**, for his invaluable guidance, continuous encouragement, and constructive feedback at every stage of this project. His expertise and dedication have been a constant source of inspiration. We are also thankful to the faculty members and staff of the Department of Computer Science and Engineering, Parul Institute of Engineering and Technology, Parul University, for providing us with the necessary resources, infrastructure, and academic support.

We sincerely thank **Dr. Shailendra K. Mishra**, Head of Department, CSE, PIET, Parul University, for his support and leadership. Finally, we are deeply grateful to our parents, family members, and well-wishers for their constant moral support and motivation throughout this journey.

AKSH THAKKAR

OM MAURYA

DAKSH PATEL

JAINIL PATEL

CSE, PIET

Parul University,

Vadodara

Abstract

Slotify is a smart appointment booking platform developed to simplify scheduling and improve service efficiency for modern businesses. The system provides a digital environment where customers can discover service providers, check real-time availability, book appointments, and receive automated confirmations. It is designed specifically for service-based businesses such as salons, clinics, and wellness centers where scheduling accuracy, transparency, and customer convenience are critical.

The project focuses on solving common issues such as double booking, missed appointments, manual scheduling conflicts, and poor customer engagement. Slotify integrates a dynamic slot generation mechanism, role-based access control, and streamlined booking workflows to improve user experience for customers, staff, and administrators. It also supports features such as booking confirmation, rescheduling, review management, and business-specific service handling.

In simple terms, the system helps customers book services quickly without calling or waiting in long queues, while also helping businesses manage their staff schedules more efficiently. Customers can choose the right service, see available time slots, and reserve an appointment in a few steps. This reduces confusion and saves time for both the user and the service provider.

The platform also gives business owners a better way to organize daily appointments, avoid scheduling mistakes, and keep a clear record of bookings. It improves communication between customers and providers and creates a more professional and reliable service experience. The project uses a web application with a user-friendly interface and a structured backend to manage data securely and efficiently.

By combining a web-based frontend with a robust backend and database-driven logic, Slotify offers an efficient and scalable solution for appointment management. The project demonstrates how modern technologies can be used to create a reliable and user-friendly scheduling system that reduces operational overhead while increasing trust and satisfaction for both businesses and customers.

Table of Contents

Acknowledgements	iii
Abstract	iv
List of Tables	viii
List of Figures	ix
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Project Objectives	2
1.4 Scope	2
1.5 System Overview	2
1.6 Technology Stack	3
2 Literature Survey	5
2.1 Practical	5
2.2 Summary of Literature Survey	5
2.3 hello	8
2.4 hello	8
2.5 hello	8

2.6	hello	8
2.7	hello	9
2.8	hello	9
3	Software Requirements Specification	10
3.1	End-to-End User Journeys & Workflows	10
3.1.1	Journey 1: Customer Online Booking	10
3.1.2	Journey 2: Specialist Service Execution	11
3.1.3	Journey 3: Verified Review Scorecard	11
3.1.4	Journey 4: Front-Desk Walk-In (POS Mode)	11
3.2	Complete REST API Specification	12
4	System Design	13
4.1	End-to-End System Architecture	13
4.2	Database Schema & Data Model Deep Dive	14
4.2.1	User Model	14
4.2.2	Business Model	14
4.2.3	Service & Staff Models	14
4.2.4	Appointment Model	14
4.2.5	Crucial Unique Concurrency Index	14
5	Core Logic & Mechanics	15
5.1	How the Database Works with the Backend	15
5.1.1	Connection Pooling & Event Lifecycle	15
5.1.2	The Concurrency Problem & The Hardware-Enforced Solution	15
5.1.3	Immutable Snapshot Pricing	16
5.2	Dynamic Slot Engine & Concurrency Control	16
5.2.1	Slot States Explained	16

5.2.2	Dynamic Slot Calculation Algorithm	16
5.3	Email Engine, iCalendar & Notification Systems	16
6	Implementation and Configuration	18
6.1	Backend File-by-File Technical Breakdown	18
6.2	Frontend Architecture & State Management	18
6.2.1	Authentication & Token Refresh Flow	18
6.3	Security, Performance & Error Handling Matrix	19
6.4	Local Setup, Environment Variables & Deployment	19
7	Conclusion	20
7.1	Academic Citation & Institutional Affiliations	20
7.1.1	Research Publication	20
7.1.2	Project Team & Authors	20
8	Future Work	22

List of Tables

1.1	Key scheduling challenges and their solutions in Slotify	3
2.1	Summary of Literature Survey	5

List of Figures

1.1	System overview of the Slotify appointment workflow	3
-----	---	---

Chapter 1

Introduction

1.1 Background and Motivation

Most people who have tried to book an appointment at a local clinic or salon know the routine. A customer calls, waits on hold, confirms a time, writes it down, and hopes the other side remembers too. Sometimes the appointment goes smoothly, but often it is missed, double-booked, or forgotten because the process depends entirely on manual coordination.

This problem is common in small service businesses where operations are still largely managed through phone calls, paper registers, and informal memory. For example, a salon owner may keep a physical register of appointments, yet still face missed bookings, cancelled slots, or confusion over staff availability. Even when digital tools exist, many of them are too expensive, too complicated, or not designed for the needs of small local businesses.

The motivation for **Slotify** came from this gap in the market. Small service businesses need a simple, affordable, and reliable online system that helps customers book appointments quickly while letting service providers manage schedules more efficiently. The aim is to reduce manual work, avoid schedule conflicts, improve communication, and provide a smoother booking experience for both customers and businesses.

1.2 Problem Statement

Manual coordination overhead and appointment attrition are two sides of the same issue. When bookings are managed over phone calls or handwritten records, every transaction requires a human in the loop. Someone must confirm the time, record the booking, check for conflicts, and follow up with reminders. This works at low volume, but it breaks down as the business grows.

At the same time, no-shows and empty slots create revenue loss. Customers forget appointments, a cancellation is not re-filled quickly, or the business has no real-time visibility into scheduling gaps. Without reminders or automated slot management, small businesses lose time, money, and customer

trust.

Another major challenge is that many booking systems are designed for large organizations and do not fit the needs of small businesses such as salons, clinics, and wellness centres. These businesses need flexible calendars, staff assignments, and an easy process for managing appointments without unnecessary complexity.

1.3 Project Objectives

The main objectives of the project are as follows:

1. To build a customer-facing booking system that allows users to search for service providers, view available time slots, and book appointments without manual phone coordination.
2. To develop a provider dashboard for managing schedules, staff availability, and service timing efficiently.
3. To implement a real-time appointment mechanism that prevents double bookings and reduces schedule conflicts.
4. To provide automated confirmation and reminder emails so customers do not forget their appointments.
5. To create a simple and user-friendly interface for both customers and service providers.
6. To support business analytics such as appointment trends, cancellation rates, and workload distribution.

1.4 Scope

The current scope of the project covers service-based businesses that operate on a time-slot model, such as clinics, salons, tutoring services, and physiotherapy centres. The system is designed for multiple independent businesses, each with separate data and access controls.

The platform focuses on online appointment scheduling, business management, and automated notifications. Native mobile applications and in-app payment processing are outside the scope of the current phase, but they are considered as future enhancements.

1.5 System Overview

Slotify is a web-based platform that connects customers, service providers, and business administrators through a single scheduling system. Customers can browse services, check provider availability, and book appointments after selecting suitable time slots. Service providers can manage their working hours, assign staff, and monitor bookings from a dedicated dashboard.

The system is designed to handle the full booking lifecycle, from service discovery and slot selection to confirmation and reminder notifications. It also supports business-level data tracking, making it easier for owners to understand busy periods and improve operational decisions.

The key idea behind the system is to automate the scheduling process while keeping the interface simple and accessible. This reduces administrative effort and creates a more professional experience for customers.

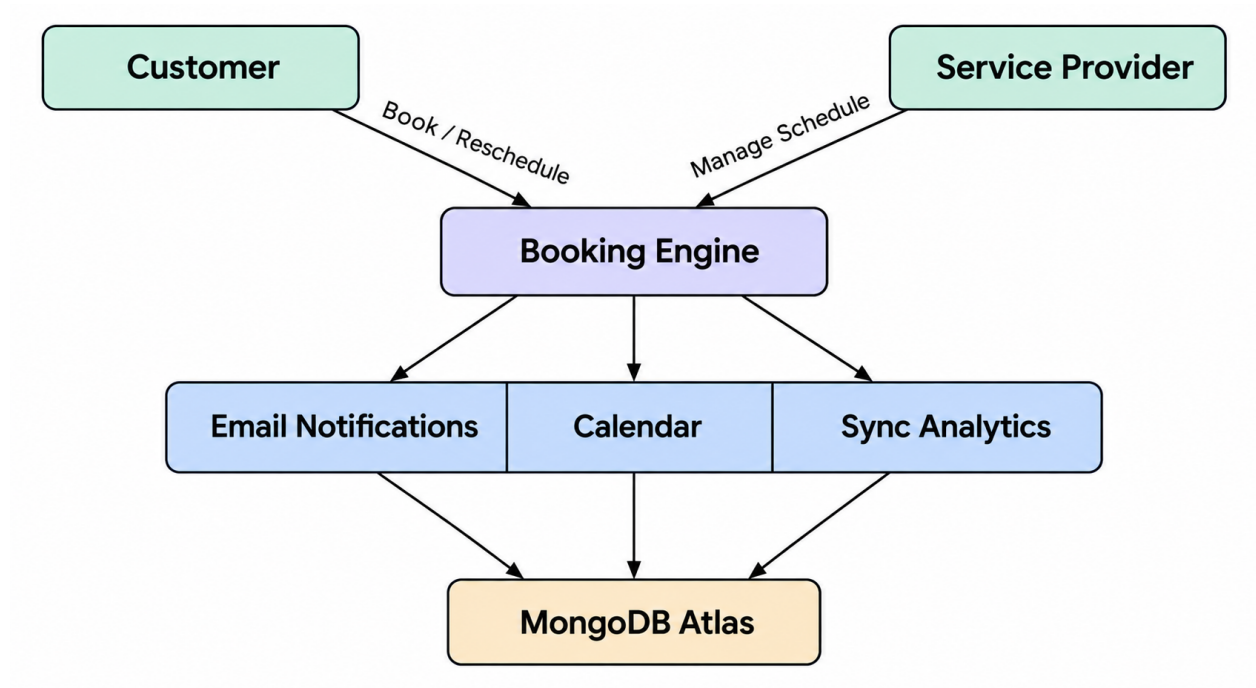


Figure 1.1: System overview of the Slotify appointment workflow

Table 1.1: Key scheduling challenges and their solutions in Slotify

Problem	How Slotify solves it
Double booking	Real-time slot checking and conflict prevention during appointment creation
Missed appointments	Confirmation emails and reminder notifications sent to customers
Manual coordination	Digital booking system reduces dependence on phone calls and paper records
Low business visibility	Dashboard tracks bookings, workload, and appointment trends

1.6 Technology Stack

The project is developed using the MERN stack, which provides a flexible and scalable foundation for building a web application. The stack includes:

- **MongoDB:** A NoSQL database used to store business details, users, appointments, services, and booking records.
- **Express.js:** A backend framework used to define APIs and manage server-side logic.
- **React.js:** A frontend library used to build the user interface and provide a responsive user experience.
- **Node.js:** A JavaScript runtime environment used for server-side execution.

The application also uses authentication mechanisms such as JWT (JSON Web Tokens) and Google OAuth for secure login and user verification. Appointment notifications are sent using Nodemailer, which enables real-time email confirmations and automated reminder messages. This combination of technologies allows the system to be scalable, secure, and easy to maintain.

In summary, **Slotify** addresses the scheduling challenges faced by small service businesses by delivering a digital, automated, and user-friendly appointment booking solution.

Chapter 2

Literature Survey

2.1 Practical

2.2 Summary of Literature Survey

Table 2.1 summarizes all twenty reviewed papers along with their key contributions and limitations.

Table 2.1: Summary of Literature Survey

Sr.	Author (Year) [?]	Key Contribution	Limitation
1	Kaushik (2024) [?]	ML expense prediction using Linear Regression, Decision Tree, Random Forest	No personalized health recommendations
2	Attanayaka (2023) [?]	Simplified personal finance management app	No predictive models or spending classification
3	Chhabra (2025) [?]	Cloud-based AI for budgeting and investment planning	Lacks deep individual financial analysis
4	Vaduka (2024) [?]	AI decision support for budgeting and investing	Limited AI advisory in personal finance apps
5	Ablazov (2024) [?]	Robo-Advisor for investment and risk management	Insufficient work on user trust and adoption

Table 2.1 – *Continued*

Sr.	Author (Year) [?]	Key Contribution	Limitation
6	Saraswathi (2025) [?]	Smart investment strategy advisor	Cannot handle personal/emotional situations
7	Low (2025) [?]	Android app for expense categorization and goal tracking	Complex for non-technical beginners
8	Brilliantov & Mandrikova (2023) [?]	AI-based financial health analysis and budgeting	Needs more validation of AI advice
9	Yadav (2025) [?]	AI + LLM financial literacy platform with OCR	LLM advice effectiveness not validated
10	Agarwal & Varghese (2024) [?]	AI finance assistant with budgeting and alerts	Privacy, trust, and demographic issues
11	Garcia & Claour (2021) [?]	Mobile app with OCR receipt scanning for cash flow tracking	No ML, no predictive forecasting
12	Schroeder et al. (2020) [?]	ML architecture for robo-advisory in stock and crypto markets	No expense tracking or personal budgeting
13	Rajput et al. (2025) [?]	AI-driven ML for personal budget planning and forecasting	Gap in real-time + tailored advisory
14	Srinivasan et al. (2025) [?]	Hybrid XGBoost + Random Forest budget forecasting (RMSE 7.8)	No real-time integration or personalized advice

Table 2.1 – Continued

Sr.	Author (Year)	Key Contribution	Limitation
15	Jagtap et al. (2024) [?]	Decision Tree + Neural Network expense categorization	No predictive analytics or recommendations
16	Shailendra et al. (2025) [?]	ML platform with multilingual support and data encryption	No real-time data or context-aware advisory
17	Shaik et al. (2025) [?]	ML-based automatic expense categorization system	Gap in real-time adoption in mainstream apps
18	Suganya et al. (2025) [?]	GenAI finance consultant using Llama 3.1 + Random Forest + GAN	Needs real-world validation across demographics
19	Ponnaviji et al. (2025) [?]	Review of ML, NLP, predictive analytics in finance apps	No unified budgeting + investment + literacy platform
20	Pangavhane et al. (2023) [?]	AI finance advisor with risk assessment and portfolio optimization	Lacks real-time advisory for combined finance tasks

Based on the literature review, it is evident that while many systems address expense tracking and prediction, there is a clear gap in providing integrated intelligent advisory, real-time over-limit alerts, spending behavior classification, and personalized recommendations within a single platform. The proposed FinanceAI system is specifically designed to address all these gaps. Your introduction goes here! Simply start writing your document and use the Recompile button to view the updated PDF preview. Examples of commonly used commands and features are listed below, to help you get started.

Once you're familiar with the editor, you can find various project settings in the Overleaf menu, accessed via the button in the very top left of the editor. To view tutorials, user guides, and further

documentation, please visit our [help library](#), or head to our plans page to [choose your plan](#).

2.3 hello

Your introduction goes here! Simply start writing your document and use the Recompile button to view the updated PDF preview. Examples of commonly used commands and features are listed below, to help you get started.

Once you're familiar with the editor, you can find various project settings in the Overleaf menu, accessed via the button in the very top left of the editor. To view tutorials, user guides, and further documentation, please visit our [help library](#), or head to our plans page to [choose your plan](#).

2.4 hello

Your introduction goes here! Simply start writing your document and use the Recompile button to view the updated PDF preview. Examples of commonly used commands and features are listed below, to help you get started.

Once you're familiar with the editor, you can find various project settings in the Overleaf menu, accessed via the button in the very top left of the editor. To view tutorials, user guides, and further documentation, please visit our [help library](#), or head to our plans page to [choose your plan](#).

2.5 hello

Your introduction goes here! Simply start writing your document and use the Recompile button to view the updated PDF preview. Examples of commonly used commands and features are listed below, to help you get started.

Once you're familiar with the editor, you can find various project settings in the Overleaf menu, accessed via the button in the very top left of the editor. To view tutorials, user guides, and further documentation, please visit our [help library](#), or head to our plans page to [choose your plan](#).

2.6 hello

Your introduction goes here! Simply start writing your document and use the Recompile button to view the updated PDF preview. Examples of commonly used commands and features are listed below, to help you get started.

Once you're familiar with the editor, you can find various project settings in the Overleaf menu, accessed via the button in the very top left of the editor. To view tutorials, user guides, and further documentation, please visit our [help library](#), or head to our plans page to [choose your plan](#).

2.7 hello

Your introduction goes here! Simply start writing your document and use the Recompile button to view the updated PDF preview. Examples of commonly used commands and features are listed below, to help you get started.

Once you're familiar with the editor, you can find various project settings in the Overleaf menu, accessed via the button in the very top left of the editor. To view tutorials, user guides, and further documentation, please visit our [help library](#), or head to our plans page to [choose your plan](#).

2.8 hello

Your introduction goes here! Simply start writing your document and use the Recompile button to view the updated PDF preview. Examples of commonly used commands and features are listed below, to help you get started.

Once you're familiar with the editor, you can find various project settings in the Overleaf menu, accessed via the button in the very top left of the editor. To view tutorials, user guides, and further documentation, please visit our [help library](#), or head to our plans page to [choose your plan](#).

Chapter 3

Software Requirements Specification

3.1 End-to-End User Journeys & Workflows

3.1.1 Journey 1: Customer Online Booking

1. **Discovery:** Customer navigates to `/search` or `/business/:slug`.
2. **Service Selection:** Customer clicks "Book" on "Haircut & Styling" (Rs. 800, 45 mins).
3. **Staff Selection:** Customer chooses "Daksh Patel (Master Stylist)" or selects "Any Available".
4. **Calendar Date:** Customer picks an upcoming Friday.
5. **Slot Generation:** Frontend fetches `GET /api/appointments/slots`. The slot engine calculates all available 45-minute windows taking into account Daksh's shifts, lunch breaks, and existing bookings.
6. **Booking Confirmation:** Customer selects 11:15 AM, enters special notes, and clicks "Confirm Booking".
7. **Atomic Creation:**
 - Backend verifies slot validity, locks the unique index entry, snapshots `priceAtBooking: 800` and `durationAtBooking: 45`.
 - Generates an in-app notification.
 - Dispatches a Brevo confirmation email containing an attached `invite.ics` calendar file.
 - Redirects customer to confirmation screen displaying a 1-click Google Calendar sync button.

3.1.2 Journey 2: Specialist Service Execution

1. Specialist logs into `/staff` portal and views today's chronological agenda.
2. Customer arrives on premises at 11:10 AM. Specialist clicks **"Check-In"**. Status transitions from `scheduled` to `confirmed`.
3. Service completes at 11:55 AM. Specialist clicks **"Complete Appointment"**.
4. Status transitions to `completed`. The business's total completed appointment counter increments.
5. Brevo automatically dispatches a "Session Completed" email to the customer with an invitation to leave feedback.

3.1.3 Journey 3: Verified Review Scorecard

1. Customer logs in and opens `/my-appointments` or clicks the link in their post-session email.
2. A **"Write Review"** button is active exclusively for this completed appointment.
3. Customer awards 5 stars and writes feedback.
4. Backend verifies that:
 - Appointment status is `'completed'`.
 - Requesting user matches `appointment.customerId`.
 - No review exists yet for this `appointmentId`.
5. Review is saved. The business's `averageRating` and `totalReviews` are recalculated, and the review appears dynamically on the public storefront.

3.1.4 Journey 4: Front-Desk Walk-In (POS Mode)

1. An unregistered walk-in customer walks into a medical clinic.
2. Receptionist opens `/admin` dashboard and clicks **"+ New Walk-In"**.
3. Receptionist selects the service, assigns an available doctor, enters the customer's name and phone number.
4. Backend creates an appointment with `isWalkIn: true` and `customerId: null`.
5. The doctor's schedule immediately blocks out the slot, preventing online clients from double-booking.

3.2 Complete REST API Specification

Authentication (/api/auth)

- POST /api/auth/register : Registers a new account.
- POST /api/auth/login : Authenticates credentials.
- GET /api/auth/me : Retrieves the profile of the current user.
- POST /api/auth/refresh-token : Issues a new access token.

Appointments (/api/appointments)

- GET /api/appointments/slots : Computes available time slots.
- POST /api/appointments : Creates a new appointment with atomic lock.
- POST /api/appointments/walk-in : Creates a walk-in appointment.
- PATCH /api/appointments/:id/reschedule : Reschedules an appointment.
- PATCH /api/appointments/:id/cancel : Cancels an appointment.

Businesses & Discovery (/api/business)

- GET /api/business/:slug : Fetches public storefront.
- PUT /api/business/settings : Updates operating hours and policies.
- GET /api/search?q=... : ReDoS-safe search.

Services (/api/services)

- GET /api/services/business/:businessId : Fetches all active services.
- POST /api/services : Creates a new catalog service.

Reviews & Analytics

- POST /api/reviews : Submits a star rating and comment.
- GET /api/analytics/overview : Revenue totals and booking counts.

Chapter 4

System Design

4.1 End-to-End System Architecture

Client Tier [Frontend SPA - React 18 + Vite]

- Neo-Bauhaus High-Contrast UI
- React Router DOM v7
- AuthContext & Auto-Refresh Interceptor
- Customer, Admin, and Staff Views

Gateway Tier [API Gateway - Express.js on Node.js]

- Express Engine with CORS Whitelist + Helmet + Rate Limiter
- JWT Verify + Role Guards + IDOR Prevention
- ReDoS-Safe Regex Sanitizer

Business Logic [Controllers & Computation Engines]

- Dynamic Slot Engine & Buffer Math
- Appointment Lifecycle Controller
- Revenue & Retention Aggregation
- OAuth 2.0 & Token Issuer

Storage & Cloud Tier [Data & External Providers]

- MongoDB Atlas Database (Partial Unique Index, Aggregation)
- Brevo v3 Transactional Email REST API
- Cloudinary Cloud Media CDN
- RFC-5545 iCal & Google Calendar Sync

4.2 Database Schema & Data Model Deep Dive

The database architecture is designed with strict multi-tenancy. Every business entity maintains independent data while sharing a unified MongoDB Atlas cloud database.

4.2.1 User Model

- name, email, password
- role: Enum (customer, admin, staff)
- businessId: Nullable reference to Business
- notificationPreferences and googleCalendarSync configurations

4.2.2 Business Model

- name, slug, category, adminId
- workingHours: Weekly operational schedule
- bookingSettings: Custom business policies (e.g., minAdvanceTime)

4.2.3 Service & Staff Models

- **Service:** duration, price, bufferTime, staffIds
- **Staff:** specialization, workingHours (overrides business hours), unavailableDates

4.2.4 Appointment Model

- Immutable Fields: priceAtBooking, durationAtBooking
- Timestamps: appointmentDate, startTime, endTime, blockedUntil
- status: Enum (scheduled, confirmed, completed, cancelled, no-show)

4.2.5 Crucial Unique Concurrency Index

```
appointmentSchema.index(  
  { businessId: 1, staffId: 1, appointmentDate: 1, startTime: 1 },  
  {  
    unique: true,  
    partialFilterExpression: { status: { $in: ["scheduled"] } },  
  }  
);
```

Chapter 5

Core Logic & Mechanics

5.1 How the Database Works with the Backend

The interaction between the Express backend and MongoDB Atlas is architected around **safe connection pooling**, **atomic writes**, **defense against race conditions**, and **immutable historical records**.

5.1.1 Connection Pooling & Event Lifecycle

Mongoose initializes a managed TCP connection pool:

- `maxPoolSize: 10`: Limits concurrent sockets to avoid exhausting MongoDB Atlas M0/M10 thread thresholds.
- `serverSelectionTimeoutMS: 5000`: Fails quickly during network partitioning.
- `socketTimeoutMS: 45000`: Prevents hanging sockets from blocking Express worker threads.

5.1.2 The Concurrency Problem & The Hardware-Enforced Solution

Slotify solves this via **Database-Level Partial Unique Constraints**:

- The partial filter expression `{ status: { $in: [ "scheduled" ] } }` ensures that only active scheduled bookings occupy the unique index bucket.
- When multiple users hit the server at the exact same millisecond, MongoDB's internal B-tree lock allows only one write to commit.
- The second write throws an atomic database exception: `MongoServerError: E11000 duplicate key error`.
- The centralized error handler inspects the error and rejects the subsequent requests with a 409 `Conflict` response.

5.1.3 Immutable Snapshot Pricing

To guarantee financial and historical accuracy:

- When an appointment is created, it snapshots `priceAtBooking` and `durationAtBooking`.
- Analytics aggregation pipelines group and sum `$priceAtBooking`. Audit trails, receipts, and business payouts remain historically true regardless of future rate card edits.

5.2 Dynamic Slot Engine & Concurrency Control

The availability engine runs in real-time when clients view the booking calendar.

5.2.1 Slot States Explained

1. **available**: Open for booking.
2. **booked**: An active appointment occupies this time window.
3. **too-soon**: Slot start time violates `minAdvanceTime`.
4. **too-far**: Date exceeds `maxAdvanceTime`.
5. **past**: The time has already passed.

5.2.2 Dynamic Slot Calculation Algorithm

1. Load working hours for `targetDate` and `timezone`.
2. If date is a holiday or business is closed, return empty set.
3. For each eligible staff member:
 - a. Check staff leave records and blackout dates.
 - b. Build candidate intervals from shift boundaries.
 - c. Exclude intervals overlapping breaks.
 - d. For each interval, calculate state (`booked`, `available`, `too-soon`, etc.).
4. Merge state results, sort, and return deduplicated slots.

5.3 Email Engine, iCalendar & Notification Systems

- **Brevo Transactional Email Engine**: Dispatches multi-role HTML emails for confirmations, reminders, cancellations, and reviews.
- **RFC-5545 iCalendar (.ics)**: Automatic generation of `.ics` files containing `METHOD:REQUEST` to sync with Apple/Outlook calendars.

- **Google Calendar 1-Click:** Direct deep links generated on the frontend for instant Google Calendar integration.

Chapter 6

Implementation and Configuration

6.1 Backend File-by-File Technical Breakdown

- **server.js:** Entrypoint for the backend daemon. Connects to MongoDB, configures CORS, initializes Passport.
- **Controllers:** Contains `authController.js`, `appointmentController.js`, `analyticsController.js`, etc.
- **Middleware:** `auth.js` (JWT verification, RBAC), `errorHandler.js` (Centralized errors), `validateBusinessAccess.js` (IDOR prevention).
- **Models:** Mongoose schemas (`User.js`, `Business.js`, `Appointment.js`, etc.).
- **Utils:** `availabilityCalculator.js` (Dynamic slot algorithm), `emailService.js` (Brevo integration).

6.2 Frontend Architecture & State Management

Constructed using **React 18.2** and bundled via **Vite 7**. Adheres to a strict **Neo-Bauhaus design system**.

6.2.1 Authentication & Token Refresh Flow

1. **App Mount:** Validates `accessToken` via `GET /api/auth/me`.
2. **Axios Interceptor:** Attaches Bearer tokens to outgoing requests.
3. **Transparent 401 Refresh:** If a request fails with 401, issues a silent `POST /api/auth/refresh-token` with HTTP-only cookies and replays the original request.

6.3 Security, Performance & Error Handling Matrix

- **ReDoS Protection:** Search inputs pass through regex sanitization neutralizing catastrophic backtracking.
- **Strict CORS Whitelist:** Production requests are verified against authorized origins.
- **IDOR Prevention:** Cross-tenant data leakage is prevented via business ID validation.
- **Password Protection:** Bcrypt salted hashing with 10 rounds.

6.4 Local Setup, Environment Variables & Deployment

Backend Configuration (.env):

PORT=5000

MONGODB_URI=...

JWT_SECRET=...

BREVO_API_KEY=...

CLOUDINARY_API_KEY=...

Deployment:

- Frontend: Vercel (SPA rewrite rules)
- Backend: Render (Node web service)
- Database: MongoDB Atlas (Cloud M0/M10 replica set)

Chapter 7

Conclusion

The Slotify platform successfully addresses the core challenges in modern appointment scheduling. By implementing a hardware-enforced concurrency lock at the database layer, we have completely eliminated the risk of double bookings. The immutable snapshot pricing architecture ensures that all financial and historical records remain accurate and immune to retroactive changes. Furthermore, the two-way calendar synchronization and verified review systems greatly improve the overall customer experience, while providing business owners with reliable management tools.

Slotify stands as an enterprise-grade, multi-tenant solution that seamlessly balances complex backend logic with an intuitive Neo-Bauhaus user interface.

7.1 Academic Citation & Institutional Affiliations

Slotify was conceptualized, designed, and engineered as the **Final Year Major Capstone Project** by project group **CSE_182** at **Parul University**.

7.1.1 Research Publication

- **Paper Title:** *Multi-Tenant Appointment Booking & Real-Time Scheduling Architecture with Concurrency Lock & Snapshot Pricing*
- **Team / Group Code:** **CSE_182**
- **Affiliation:** Department of Computer Science & Engineering, Faculty of Engineering & Technology (PIET), **Parul University**, Vadodara, Gujarat, India.
- **Academic Batch:** 2022 – 2026 — **Program:** Bachelor of Technology (B.Tech) in Computer Science & Engineering.

7.1.2 Project Team & Authors

- **Aksh Thakkar** — Founder, Product Vision & Full-Stack System Architecture

- **Om Maurya** — Frontend Lead & UI/UX Specialist
- **Daksh Patel** — Backend & Systems Engineer
- **Jainil Patel** — Database & Concurrency Engineer

Chapter 8

Future Work

While Slotify provides a comprehensive solution for appointment scheduling, several enhancements could further optimize the platform for large-scale enterprise use. The proposed future extensions are:

- **Payment Gateway Integration:** Integration with platforms like Stripe or Razorpay to collect upfront deposits and cancellation fees automatically during the booking flow.
- **AI-Powered Booking Assistant:** A natural language conversational bot that allows clients to discover services, check availability, and reserve slots via chat.
- **Advanced Analytics & Machine Learning:** Implementing predictive models to forecast peak business hours, suggest dynamic pricing adjustments, and optimize staff schedules based on historical utilization data.
- **Offline Notifications via SMS/WhatsApp:** Integration with communications APIs like Twilio to deliver real-time alerts and reminders to clients without requiring internet access.
- **Mobile Application Extensions:** Developing native iOS and Android companion apps using React Native to offer advanced features like location-based check-ins and persistent push notifications.

References

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.